

CAMPUSFLOW

Smart Campus Resource & Event Management System

Programming in Java — CSE2006

STUDENT NAME	Animesh Arjariya
REGISTRATION NO.	25BAI11301
FACULTY / MENTOR	Dr Manorama Chouhan
INSTITUTION	VIT Bhopal University
SUBMISSION DATE	18 September 2026

FINAL ACADEMIC PROJECT REPORT

Prepared for the CSE2006 Programming in Java course. The report documents the system design, Java implementation, testing, database integration and functional validation of CampusFlow.

VIT Bhopal University

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Java Syllabus Coverage
11. Screenshots and Results
12. Testing and Verification
13. Challenges Faced
14. Learnings and Key Takeaways
15. Future Enhancements
16. References

1. Introduction

CampusFlow is a modular Java console application designed to manage shared campus resources and campus events. It combines resource discovery, role-based access, conflict-aware booking, event registration, notifications, reporting, file export and runtime diagnostics in a single application.

The project is intentionally designed so that Java language features are used as part of a coherent information system rather than as isolated classroom examples. The model layer demonstrates object-oriented design, the service layer contains business rules, repository classes handle persistence, and supporting services demonstrate JDBC, I/O, reflection and multithreading.

2. Problem Statement

Campus facilities such as classrooms, laboratories, seminar spaces and equipment are shared by students, faculty members and departments. Manual or fragmented booking processes can make availability difficult to track, create conflicting reservations and make usage records harder to maintain.

CampusFlow addresses this problem through a Java-based management system that provides user management, resource discovery, conflict-aware booking, event registration, notifications and reporting in one application.

3. Objectives

- Apply Java fundamentals and flow control in a practical application.
- Demonstrate encapsulation, inheritance, abstraction and polymorphism.
- Use method overloading, overriding, interfaces, enums and nested or anonymous classes.
- Implement exception handling with custom exception types.
- Demonstrate multithreading and synchronization in booking-related processing.
- Use collections, arrays, recursion and Java I/O streams where they have a meaningful role.
- Use JPA/Hibernate for object-relational persistence and JDBC for direct SQL reporting.
- Maintain a modular project structure suitable for version control and academic evaluation.

4. Functional Requirements

Module	Requirement
User Management	Student registration, login and role-specific dashboards.
Resource Management	List resources and allow administrators to add classrooms, laboratories and equipment.
Booking Management	Create, view and cancel bookings with date/time validation.
Conflict Detection	Reject overlapping bookings for the same resource and date.
Event Management	Create and list campus events and allow users to register.
Notifications	Display booking and event confirmation messages.
Reporting	Generate user activity and resource usage reports.
File Export	Write user activity reports to text files using Java I/O.
Diagnostics	Inspect annotated service methods at runtime using reflection.
JDBC Reporting	Allow administrators to retrieve a booking count using direct SQL.

5. Non-Functional

Requirement	CampusFlow approach
Performance	Simple repository queries and collection traversal keep the console workflow lightweight.
Reliability	Validation and custom exceptions prevent invalid operations from terminating the application.
Maintainability	Models, repositories, services, I/O, JDBC, threading and utilities are separated into packages.
Scalability	Abstract user/resource hierarchies allow additional roles or resource types to be introduced.
Usability	Role-specific menus and explicit prompts provide a straightforward console workflow.
Resource Efficiency	The background monitor sleeps between checks and can be shut down cleanly
Security consideration	

Requirements

The current academic prototype uses local credentials and is not presented as production-grade authentication.

6. System Architecture

CampusFlow uses a layered architecture that separates user interaction, business rules and persistence. Supporting services provide direct database reporting, file output, runtime reflection and background processing.

```
+-----+
| Presentation: Main / Menu |
+-----+
      |
v
+-----+
| Services: Auth / Booking |
| Event / Report / Notify |
+-----+
      |
v
+-----+
| Repositories: User /      |
| Resource / Booking / Event |
+-----+
      |
v
+-----+
| JPA / Hibernate / JDBC   |
+-----+
      |
v
+-----+
| H2 Local Database        |
+-----+

Supporting services:
FileReportWriter | BookingMonitor | ReflectionService
```

This structure keeps the console interface independent of persistence details. BookingService owns the core availability rule and coordinates repositories and notifications. JPA provides entity persistence, while JdbcReportService demonstrates direct SQL access.

7. Design Diagrams

7.1 Use Case Overview

```
Student  -> Login, Browse Resources, Book, Cancel, View Events,
          Register Event, Export Activity, Usage Report
Faculty  -> Login, Browse Resources, Book, Cancel, View Events,
          Register Event
Admin    -> Login, Resource/Event Management, Reports,
          JDBC Booking Count, Diagnostics
```

7.2 Booking Workflow

```
Start -> Login/Register -> Authenticate -> Identify Role
      -> Select Operation -> Check Availability
      -> [Available: Save Booking -> Notify]
      -> [Conflict: Reject Booking]
      -> Dashboard -> Logout -> End
```

7.3 Class Hierarchy

```
User (abstract)
|-- Student
|-- Faculty
|-- Admin

Resource (abstract)
|-- Classroom
|-- Laboratory
|-- Equipment
```

7.4 Booking Sequence

```
User -> BookingService -> ResourceRepository -> H2
      -> BookingRepository -> H2
      -> overlap check
      -> save booking
      -> NotificationService -> User
```

7.5 ER Design

```
USERS 1 ---- M BOOKINGS M ---- 1 RESOURCES
USERS 1 ---- M EVENT_REGISTRATIONS M ---- 1 EVENTS
USERS 1 ---- M NOTIFICATIONS
```

The complete textual UML-style diagrams and data-flow diagrams are maintained in docs/diagrams.md in the submission bundle.

8. Design Decisions and Rationale

- A console interface keeps the project focused on Java programming concepts instead of a UI framework.
- JPA/Hibernate is used for object-relational persistence, keeping entity classes close to the domain model.
- JDBC is used separately for a direct SQL reporting feature so both persistence approaches required by the course are visible.
- BookingService synchronizes booking creation so the availability check and save operation are protected as one critical section.
- Custom exceptions make validation and booking conflicts explicit to the caller.
- NotificationService is an interface so the notification implementation can be replaced without changing booking logic.
- Reflection and @Auditable provide a meaningful runtime diagnostics feature rather than a standalone language demonstration.
- The background BookingMonitor demonstrates a thread lifecycle and clean shutdown without continuously cluttering the user-facing console.

9. Implementation Details

The source tree is divided into annotation, exception, I/O, JDBC, model, reflection, repository, service, thread, UI and utility packages. This package structure supports separation of responsibilities and makes individual Java concepts easy to locate during a viva.

Domain Model

User is an abstract JPA entity with Student, Faculty and Admin subclasses. Resource is an abstract JPA entity with Classroom, Laboratory and Equipment subclasses. Booking, CampusEvent, EventRegistration and Notification represent transactional application data.

Booking Logic

BookingService validates required values, verifies that the end time follows the start time, rejects past dates, checks resource availability, retrieves existing bookings for the resource/date, detects time overlap, saves the booking and sends a confirmation notification.

Persistence

The application uses an H2 local database through Hibernate/JPA repositories. JdbcReportService additionally demonstrates DriverManager, PreparedStatement and ResultSet for a direct booking-count query.

Threading

BookingMonitor extends Thread, increments an internal health-check counter and sleeps between checks. Its shutdown method sets a volatile flag and interrupts the thread so the application can terminate cleanly.

I/O and Reflection

FileReportWriter writes activity reports using character and byte-oriented stream APIs. ReflectionService inspects BookingService and can identify methods carrying the Auditable annotation.

10. Java Syllabus Coverage

Syllabus concept	CampusFlow implementation
Java basics and flow control	Console input, loops, switch, validation and operators
Classes, objects, methods, constructors	Model, repository and service classes
Encapsulation	Private fields and controlled accessors
Inheritance	Student/Faculty/Admin and Classroom/Laboratory/Equipment hierarchies
Overloading	BookingService and InputValidator
Overriding	User and Resource subclass behaviour
Abstract classes	User and Resource
Interfaces	NotificationService
Runtime polymorphism	User and Resource references
this / super / final	Constructors and final service references
Enums	Roles, resource types, booking status and event categories
Nested / anonymous classes	Booking.Builder and anonymous notification implementation

Singleton-style design	JPAUtil
Exceptions	CampusException hierarchy and validation
Annotations / Reflection	@Auditable and ReflectionService
Multithreading / synchronization	BookingMonitor and synchronized booking creation
Collections	ArrayList, Vector, Stack and Map
Arrays / recursion	Report arrays and ResourceHierarchy traversal
I/O streams	FileReportWriter
JDBC	JdbcReportService
JPA / ORM / JPQL	Entities, repositories and persistence queries

11. Screenshots and Results

The following captures were produced during the local validation session. They demonstrate the main console workflow and successful functional operations. Some captures were taken before the

The following captures were produced during local validation and demonstrate the main, background-monitor console output was cleaned up; the final source removes that repeated console console workflow and successful functional operations, including role-based access, noise while retaining the thread.
resource browsing, booking and reporting.

Main application menu and role entry points

```
=====
                        CAMPUSFLOW
=====
1. Login
2. Register Student
3. Browse Resources
4. View Events
5. System Diagnostics
6. Exit
Choice: [Background Monitor] System check at 19:13:39
[Background Monitor] System check at 19:13:44
[Background Monitor] System check at 19:13:49
```

Main application menu and role entry points.

Student dashboard with resource, booking, event and reporting options

```
=====
                        STUDENT DASHBOARD
=====
1. View Resources
2. Book Resource
3. My Bookings
4. Cancel Booking
5. View Events
6. Register for Event
7. Export My Activity
8. View Usage Report
0. Logout
Choice: 1[Background Monitor] System check at 18:57:19
[Background Monitor] System check at 18:57:24
[Background Monitor] System check at 18:57:29
█
```

Student dashboard with resource, booking, event and reporting options.

Conflict detection rejecting a second booking for an occupied time slot

```
=====
                        AVAILABLE RESOURCES
=====
1 | AR-202 | CLASSROOM | Academic Block | capacity=60 | available=true
2 | AI Lab | LABORATORY | AB-218 | capacity=40 | available=true
3 | Projector-01 | EQUIPMENT | Central Store | capacity=1 | available=true
Resource ID: 1
Date (YYYY-MM-DD): [Background Monitor] System check at 19:06:12
[Background Monitor] System check at 19:06:17
2026-09-17[Background Monitor] System check at 19:06:22

Start time (HH:MM): [Background Monitor] System check at 19:06:27
[Background Monitor] System check at 19:06:32
10:00
End time (HH:MM): [Background Monitor] System check at 19:06:37
[Background Monitor] System check at 19:06:42
11:00
Operation failed: The resource is already booked during the selected time.
```

Conflict detection rejecting a second booking for an occupied time slot.

Successful AR-202 booking with confirmation notification

```
Start time (HH:MM): [Background Monitor] System check at 18:59:49
[Background Monitor] System check at 18:59:54
[Background Monitor] System check at 18:59:59
[Background Monitor] System check at 19:00:04
[Background Monitor] System check at 19:00:09
[Background Monitor] System check at 19:00:14
[Background Monitor] System check at 19:00:19
[Background Monitor] System check at 19:00:24
10:00
End time (HH:MM): [Background Monitor] System check at 19:00:30
11:00[Background Monitor] System check at 19:00:35

[Notification] Booking confirmed for AR-202 on 2026-09-17
Booking created: 1 | resource=1 | date=2026-09-17 | 10:00-11:00 | CONFIRMED
```

Successful AR-202 booking with confirmation notification.

Campus event listing and successful event registration

```
Choice: [Background Monitor] System check at 19:08:12
[Background Monitor] System check at 19:08:17
[Background Monitor] System check at 19:08:22
[Background Monitor] System check at 19:08:27
[Background Monitor] System check at 19:08:32
[Background Monitor] System check at 19:08:37
[Background Monitor] System check at 19:08:42
[Background Monitor] System check at 19:08:47
[Background Monitor] System check at 19:08:52
6

=====
                CAMPUS EVENTS
=====
1 | Java Project Showcase | TECHNICAL | 2026-09-23 | Seminar Hall | capacity=100
Event ID: [Background Monitor] System check at 19:08:57
1

[Notification] Registration confirmed for Java Project Showcase
Event registration successful.
```

Campus event listing and successful event registration.

Administrator dashboard and direct JDBC booking-count result

```
=====
ADMINISTRATOR DASHBOARD
=====
1. View Resources
2. Book Resource
3. My Bookings
4. Cancel Booking
5. View Events
6. Register for Event
7. Export My Activity
8. View Usage Report
9. Add Resource
10. Add Event
11. JDBC Booking Count
0. Logout
Choice: [Background Monitor] System check at 19:20:34
[Background Monitor] System check at 19:20:39
11
JDBC booking count: 1
```

Administrator dashboard and direct JDBC booking-count result.

12. Testing and Verification

Automated verification is provided through JUnit 5. The current test class covers booking conflict detection and polymorphic resource-cost behaviour. The local Maven build performed by the student also completed compilation and packaging successfully.

Check	Result
Maven clean test	BUILD SUCCESS
JUnit tests	2 run; 0 failures; 0 errors; 0 skipped
Maven clean package	BUILD SUCCESS
Compiled source files	41
Packaged artifact	target/campusflow-1.0.0.jar
Java target	release 17

The package build produced target/campusflow-1.0.0.jar. The Java 17 release target was configured in pom.xml, which removes the earlier source/target system-module warning when using a newer installed JDK.

Recommended Manual Test Cases

Case	Expected result
Valid login	Role-specific dashboard is displayed.
Invalid login	Login failure message is shown.
Valid booking	Booking is stored and notification is displayed.
Overlapping booking	BookingConflictException is handled and booking is rejected.
Invalid time range	Validation message is displayed.
Event registration	Registration is stored and confirmation is displayed.

Report export	Text report is created under reports/.
Admin JDBC count	Database booking count is returned.
Diagnostics	Reflection output and annotation information are displayed.

13. Challenges Faced

- Separating persistence operations from business rules while keeping the project understandable.
- Preventing conflicting bookings when multiple operations can access the same resource.
- Handling invalid console input without terminating the application.
- Coordinating JPA persistence with a separate JDBC reporting feature.
- Keeping the multithreading demonstration useful without flooding the interactive console with repeated status messages.

14. Learnings and Key Takeaways

The project demonstrates that Java concepts become more useful when combined around a real problem. Inheritance and polymorphism reduce duplication in domain models, interfaces allow services to be replaced, custom exceptions keep failure cases explicit, synchronization protects a critical booking operation, collections simplify dynamic data handling, JPA maps domain objects to relational data, JDBC provides direct SQL access, and I/O makes reports available outside the database.

The modular package structure also provides a practical example of maintainability: a change to the notification implementation does not require rewriting the booking rules, and database access is isolated from the console menus.

15. Future Enhancements

- JavaFX or web-based graphical interface.
- Stronger password hashing and production-grade authentication.
- Email or push notifications.
- Recurring bookings and calendar integration.
- Richer analytics and visual usage dashboards.
- Administrator audit dashboard and more granular permissions.
- Database migration scripts and deployment configuration.
- Role-based authorization rules beyond menu visibility.

16. References

1. CSE2006 Programming in Java course syllabus supplied for the project.
2. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.
3. Herbert Schildt, Java: The Complete Reference, 11th Edition.
4. Oracle Java documentation and Jakarta Persistence documentation consulted during implementation.